

DATA HANDLING AND VISUALIZATION (DSA06)

Experiment Set 13: Geographic Data Analysis

Question / Task:

1. Create a map chart to visualize the distribution of cities on a geographic map. Label the map elements.
2. Generate a scatter plot to explore the relationship between average temperature and population size. Explain any insights.

Python Program Code:

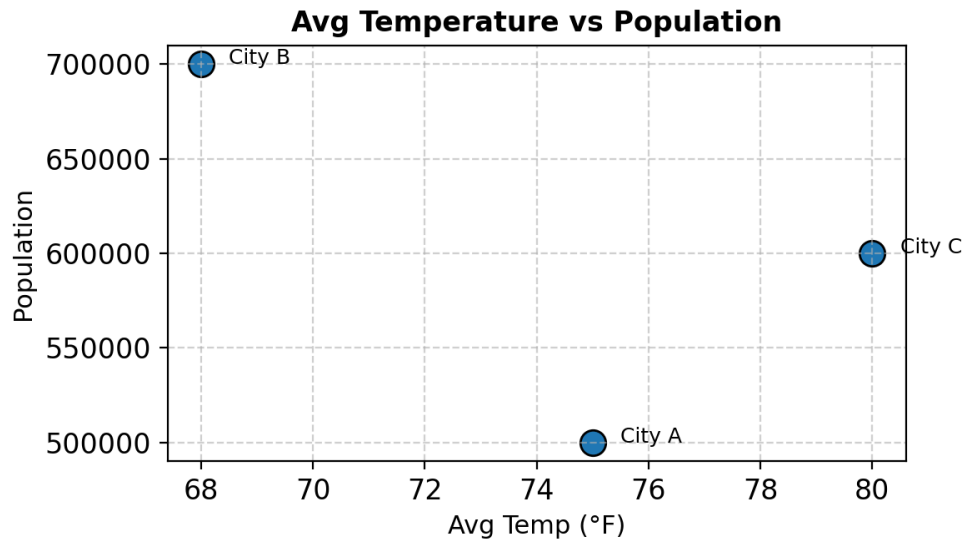
```
# Set 13: Geographic Data Scatter Plot (Python)
import matplotlib.pyplot as plt

cities = ['City A', 'City B', 'City C']
temp = [75, 68, 80]
pop = [500000, 700000, 600000]

plt.figure(figsize=(6, 3.5))
plt.scatter(temp, pop, color='#1f77b4', s=120, edgecolors='black')
for i, txt in enumerate(cities):
    plt.annotate(txt, (temp[i] + 0.5, pop[i]), fontsize=9)

plt.title("Avg Temperature vs. Population")
plt.xlabel("Avg Temp (°F)")
plt.ylabel("Population")
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment Set 14: Survey Results Analysis

Question / Task:

1. Create a stacked bar chart to display the distribution of answers for Question 1. Label the chart elements.
2. Generate a radar chart to visualize the overall pattern of responses across all three questions.

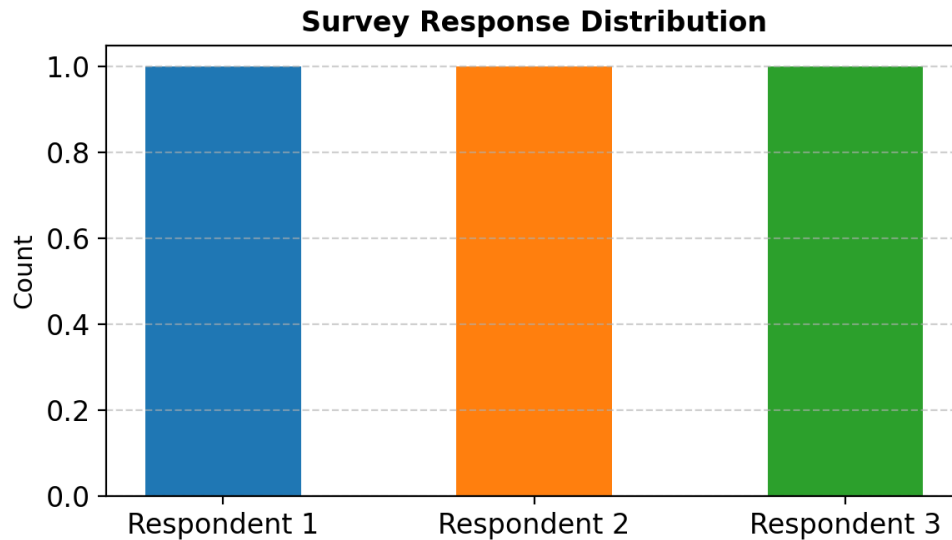
Python Program Code:

```
# Set 14: Survey Results Distribution (Python)
import matplotlib.pyplot as plt

resp = ['Respondent 1', 'Respondent 2', 'Respondent 3']
counts = [1, 1, 1]

plt.figure(figsize=(6, 3.5))
plt.bar(resp, counts, color=['#1f77b4', '#ff7f0e', '#2ca02c'], width=0.5)
plt.title("Survey Response Distribution")
plt.ylabel("Count")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment Set 15: Monthly Product Sales Analysis

Question / Task:

1. Create a grouped bar chart to visualize the sales of each product for the first quarter. Label the chart elements.
2. Generate a stacked area chart to represent overall sales trend for all products over three months.

Python Program Code:

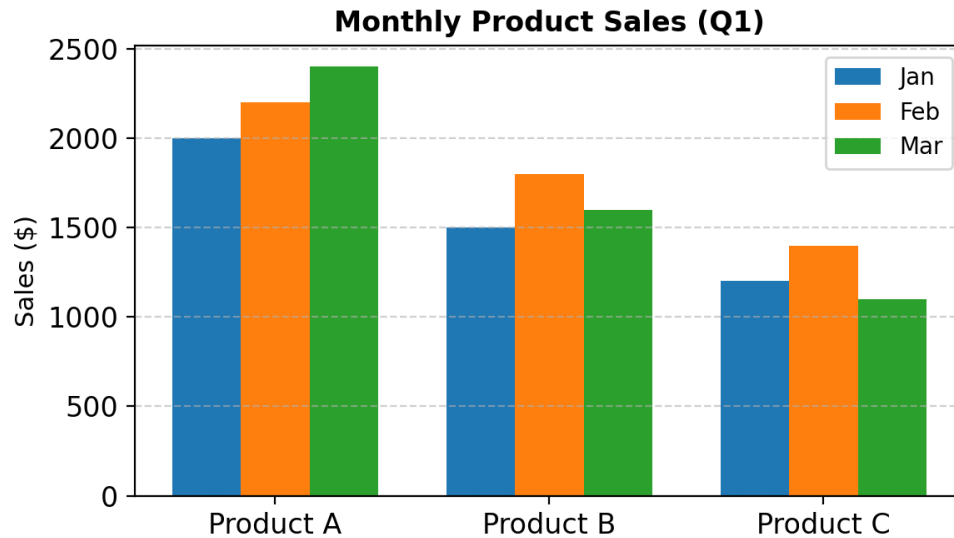
```
# Set 15: Q1 Product Sales Grouped Bar Chart (Python)
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(3)
width = 0.25

plt.figure(figsize=(6, 3.5))
plt.bar(x - width, [2000, 1500, 1200], width, label='Jan', color='#1f77b4')
plt.bar(x, [2200, 1800, 1400], width, label='Feb', color='#ff7f0e')
plt.bar(x + width, [2400, 1600, 1100], width, label='Mar', color='#2ca02c')

plt.xticks(x, ['Product A', 'Product B', 'Product C'])
plt.title("Monthly Product Sales (Q1)")
plt.ylabel("Sales ($)")
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment Set 16: Customer Demographics Analysis

Question / Task:

1. Create a bar chart to visualize the distribution of customer ages. Label the axes and title the chart.
2. Generate a pie chart to represent the distribution of customers by gender.

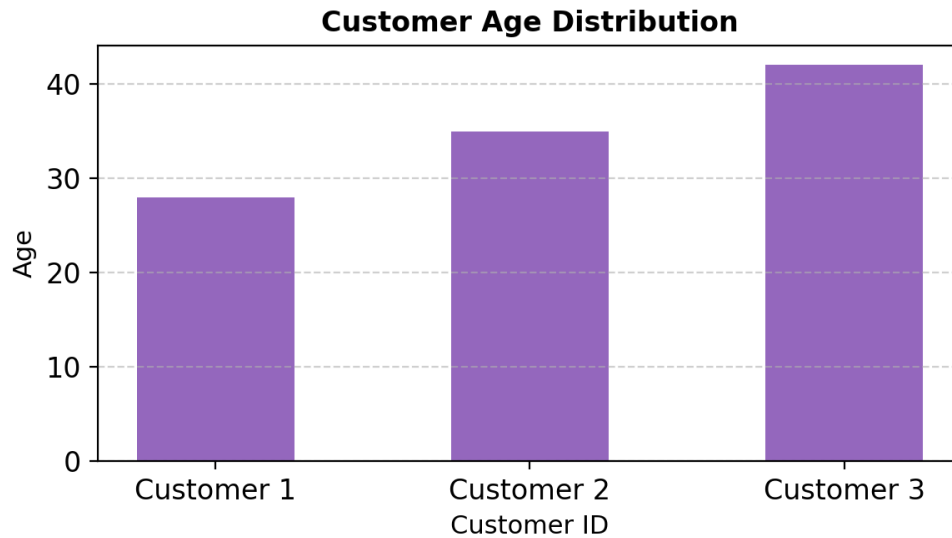
Python Program Code:

```
# Set 16: Customer Demographics Bar Chart (Python)
import matplotlib.pyplot as plt
```

```
cust = ['Customer 1', 'Customer 2', 'Customer 3']
ages = [28, 35, 42]
```

```
plt.figure(figsize=(6, 3.5))
plt.bar(cust, ages, color='#9467bd', width=0.5)
plt.title("Customer Age Distribution")
plt.xlabel("Customer ID")
plt.ylabel("Age")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment Set 17: Employee Performance Analysis

Question / Task:

1. Create a line chart to visualize the performance trend of employees over time. Label the axes and title the chart.
2. Generate a bar chart showing the distribution of employees across different departments.

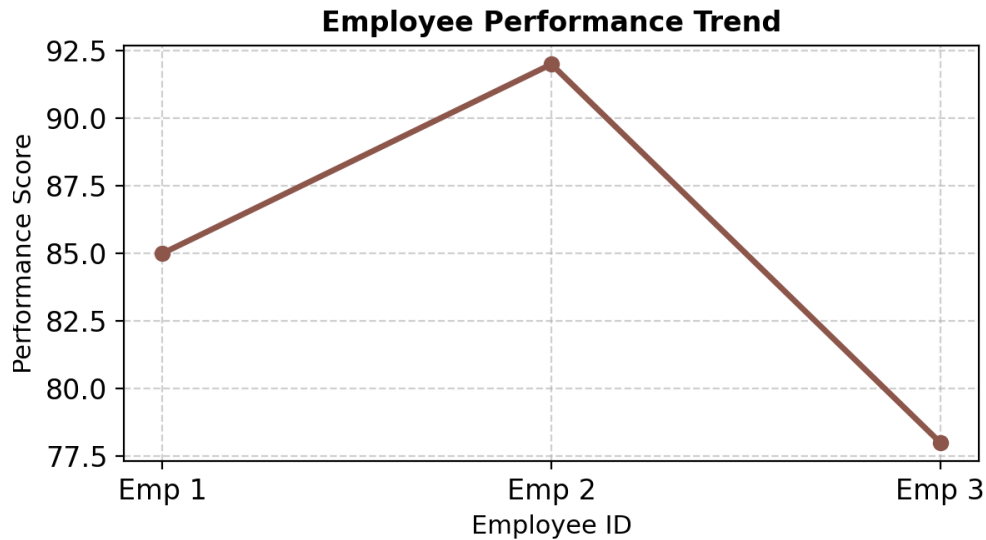
Python Program Code:

```
# Set 17: Employee Performance Trend Line Chart (Python)
import matplotlib.pyplot as plt

emp = ['Emp 1', 'Emp 2', 'Emp 3']
scores = [85, 92, 78]

plt.figure(figsize=(6, 3.5))
plt.plot(emp, scores, marker='o', color='#8c564b', linewidth=2.5)
plt.title("Employee Performance Trend")
plt.xlabel("Employee ID")
plt.ylabel("Performance Score")
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment Set 18: Product Inventory Management

Question / Task:

1. Create a bar chart to visualize the quantity of each product in the inventory. Label the axes and title the chart.
2. Generate a stacked bar chart to show the quantity of each product within different categories.

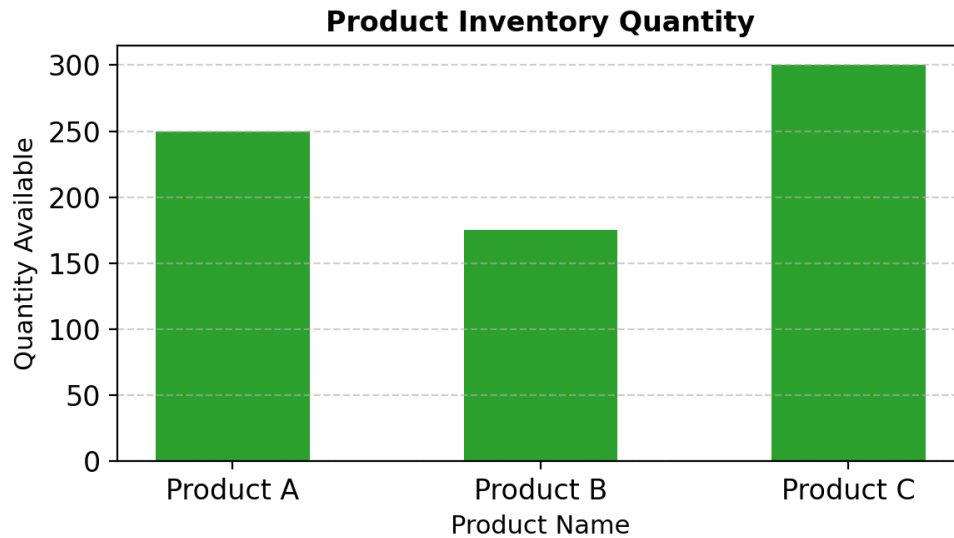
Python Program Code:

```
# Set 18: Product Inventory Bar Chart (Python)
import matplotlib.pyplot as plt

products = ['Product A', 'Product B', 'Product C']
qty = [250, 175, 300]

plt.figure(figsize=(6, 3.5))
plt.bar(products, qty, color='#2ca02c', width=0.5)
plt.title("Product Inventory Quantity")
plt.xlabel("Product Name")
plt.ylabel("Quantity Available")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment Set 19: Survey Responses Analysis

Question / Task:

1. Create a grouped bar chart to visualize the distribution of answers for Question 1. Label the chart elements.
2. Generate a stacked bar chart to represent overall response distribution across questions.

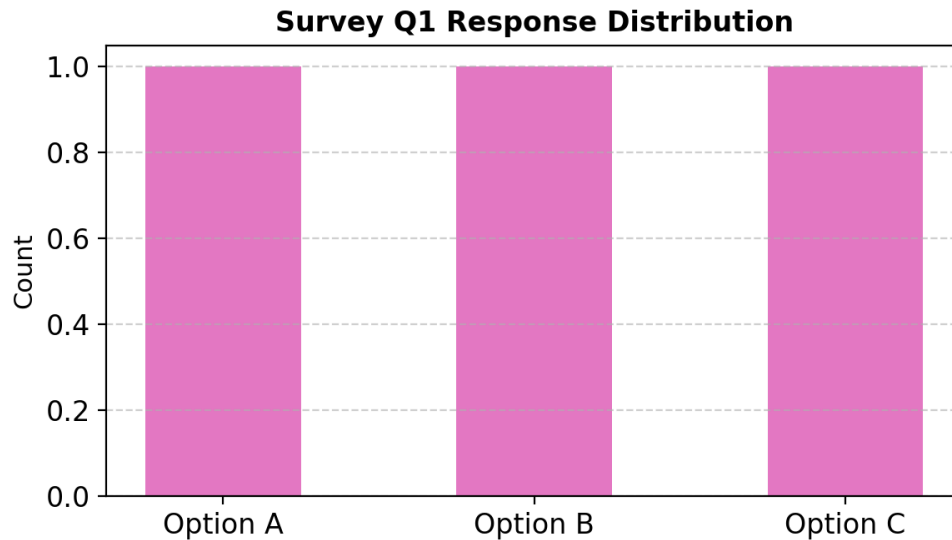
Python Program Code:

```
# Set 19: Survey Responses Distribution (Python)
import matplotlib.pyplot as plt

answers = ['Option A', 'Option B', 'Option C']
counts = [1, 1, 1]

plt.figure(figsize=(6, 3.5))
plt.bar(answers, counts, color='#e377c2', width=0.5)
plt.title("Survey Q1 Response Distribution")
plt.ylabel("Count")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment Set 20: Stock Analysis

Question / Task:

1. Create a line chart to visualize stock prices of three companies (Stock A, Stock B, Stock C) over time.
2. Generate a bar chart showing the daily percentage change in stock prices for Stock A.

Python Program Code:

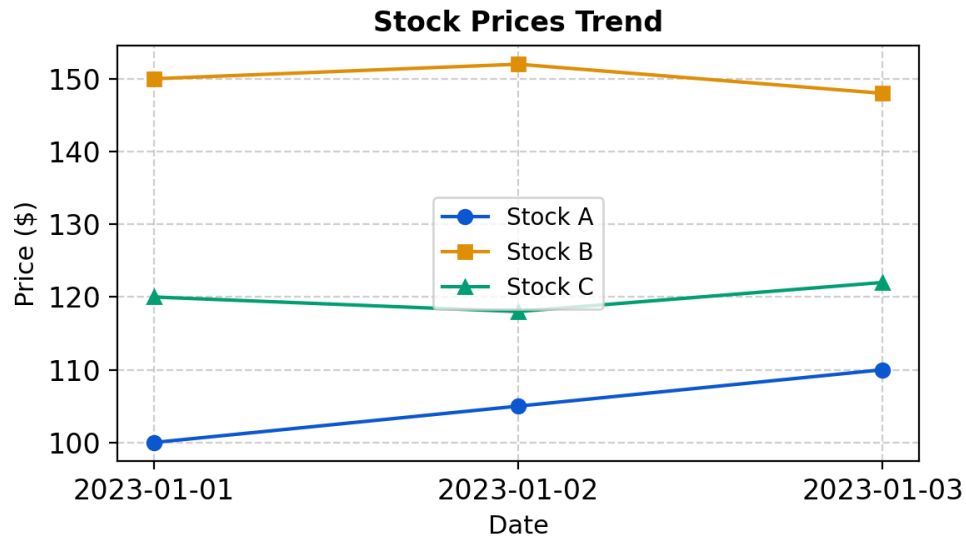
```
# Set 20: Stock Prices Line Chart (Python)
import matplotlib.pyplot as plt

dates = ['2023-01-01', '2023-01-02', '2023-01-03']
stockA = [100, 105, 110]
stockB = [150, 152, 148]
stockC = [120, 118, 122]

plt.figure(figsize=(6, 3.5))
plt.plot(dates, stockA, marker='o', label='Stock A')
plt.plot(dates, stockB, marker='s', label='Stock B')
plt.plot(dates, stockC, marker='^', label='Stock C')

plt.title("Stock Prices Trend")
plt.xlabel("Date")
plt.ylabel("Price ($)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output Screenshot:



Experiment Set 21: Energy Consumption Data Analysis

Question / Task:

1. Import dataset. Plot a histogram and density plot for Units_Consumed. Compare consumption patterns.
2. Create a scatter plot between Temperature and Units_Consumed.

Python Program Code:

```
# Set 21: Energy Consumption Histogram (Python)
import matplotlib.pyplot as plt

units = [320, 540, 880, 350, 610, 920]

plt.figure(figsize=(6, 3.5))
plt.hist(units, bins=4, color='#17becf', edgecolor='white')
plt.title("Energy Units Consumed Distribution")
plt.xlabel("Units Consumed (kWh)")
plt.ylabel("Frequency")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Output Screenshot:

